

AI-Powered Stock Screener

Complete Technical Documentation

1. Project Overview

1.1 Introduction

AI-Powered Stock Screener is an end-to-end intelligent financial query platform that enables users to interact with stock market and company financial data using natural language.

Instead of manually writing SQL queries or using rigid dashboard filters, users can simply type queries such as:

- "Technology companies with PE ratio less than 20"
- "Companies with revenue growth in Q2 2024"
- "Companies with high revenue growth and low PE ratio"

The system processes the query through multiple backend layers:

1. Natural Language Understanding Layer (LLM)
2. DSL Translation Layer
3. DSL Validation Layer
4. SQL Compilation Layer
5. Query Execution Layer
6. Result Formatting Layer
7. Frontend Presentation Layer

The system was designed as a modular architecture to ensure:

- Scalability
 - Security
 - Query Safety
 - Extensibility
 - Maintainability
 - AI integration capability
-

2. Project Objectives

2.1 Primary Objectives

The primary objectives of the project are:

- Build a natural language driven financial screening system

- Reduce dependency on manual SQL queries
 - Create safe database access using controlled DSL
 - Implement modular backend architecture
 - Build real-time query execution pipeline
 - Support analytical stock filtering
 - Integrate AI/LLM into backend systems
-

2.2 Secondary Objectives

- Portfolio management
 - Alert management system
 - Financial metric filtering
 - Quarter-based analysis
 - Growth-based filtering
 - Query validation and guardrails
 - API-driven frontend integration
-

3. Technology Stack

3.1 Backend Stack

Component	Technology
Backend Framework	FastAPI
Programming Language	Python
ORM / SQL Layer	SQLAlchemy
Authentication	JWT
Validation	Pydantic
AI Integration	Google Gemini API
Caching	Redis
API Testing	Swagger UI

3.2 Frontend Stack

Component	Technology
Frontend Framework	Streamlit
Visualization	Pandas + Streamlit Charts
Session Handling	Streamlit Session State

3.3 Database Stack

Component	Technology
Database	PostgreSQL
Query Engine	SQL
Data Storage	Relational Tables

3.4 Planned DevOps Stack

The project is designed to later support DevOps integration.

Layer	Tool
Containerization	Docker
Orchestration	Kubernetes / Amazon EKS
CI/CD	Jenkins
Infrastructure as Code	Terraform
Monitoring	Prometheus
Visualization	Grafana
Cloud Deployment	AWS

4. High-Level Architecture

4.1 System Flow

```
User
↓
Streamlit Frontend
↓
FastAPI Backend
↓
LLM Parser Layer
↓
DSL Generator
↓
DSL Validation Layer
↓
SQL Compiler
```

↓
Execution Layer
↓
PostgreSQL Database
↓
JSON API Response
↓
Frontend Result Rendering

4.2 Architectural Philosophy

The architecture follows a modular layered design.

Each layer has a dedicated responsibility.

This separation improves:

- Debugging
 - Maintainability
 - Security
 - Future scalability
 - AI integration
 - DevOps compatibility
-

5. Low-Level System Design

5.1 Frontend Layer

Purpose

The frontend acts as the user interaction layer.

It allows:

- Natural language query input
 - Result visualization
 - Portfolio management
 - Alert management
 - Company exploration
 - Session handling
-

Framework

Frontend is implemented using Streamlit.

Reason for selection:

- Fast prototyping
 - Easy Python integration
 - Built-in charts and tables
 - Simplified state management
 - Suitable for data-driven applications
-

Frontend Screens

1. AI Screener Screen

Features

- Natural language query input
- Query suggestions
- Search execution
- Result tables
- Pagination
- Metrics visualization
- Charts
- Quick query buttons

Components

- Input Box
 - Search Button
 - Results Table
 - Charts
 - Metrics Cards
 - Sidebar Navigation
-

2. Portfolio Screen

Features

- Add stock to portfolio
- Portfolio listing
- Profit/loss calculation
- Investment tracking

Components

- Stock symbol input

- Quantity input
 - Portfolio table
 - Delete actions
-

3. Watchlist Screen

Features

- Add companies to watchlist
 - Remove companies
 - View tracked companies
-

4. Alerts Screen

Features

- Create alerts
 - Define conditions
 - Monitor metrics
 - Delete alerts
-

5. Company Explorer Screen

Features

- Company overview
 - Financial metrics
 - Price history
 - Trend charts
 - Financial analysis
-

Session Management

Implemented using Streamlit Session State.

Maintains:

- JWT token
 - Username
 - Search results
 - Query history
 - Current page state
 - Selected company
-

5.2 Backend Layer

Framework

FastAPI

Why FastAPI?

- High performance
 - Asynchronous support
 - Automatic Swagger documentation
 - Pydantic integration
 - API-first design
 - Easy JWT integration
-

Backend Responsibilities

- API routing
 - Authentication
 - Query orchestration
 - DSL validation
 - SQL compilation
 - Database communication
 - Error handling
 - Response formatting
-

5.3 Authentication Layer

Authentication Method

JWT-based authentication.

Features

- User registration
 - User login
 - Password hashing
 - Token generation
 - Route protection
 - Token validation
-

Security Measures

- Password hashing
 - Bearer token authorization
 - Protected endpoints
 - Session validation
-

5.4 LLM Parser Layer

Purpose

Convert natural language financial queries into structured DSL JSON.

LLM Used

Google Gemini API.

Example Input

Technology companies with PE ratio less than 30

Example DSL Output

```
{
  "filters": [
    {
      "field": "sector",
      "operator": "=",
      "value": "Technology"
    },
    {
      "field": "pe_ratio",
      "operator": "<",
      "value": 30
    }
  ],
  "logic": "AND"
}
```

Prompt Engineering Strategy

The parser prompt enforces:

- JSON-only responses
 - Controlled fields
 - Controlled operators
 - DSL schema compliance
 - No additional explanation text
-

Challenges Faced

- Invalid JSON responses
 - Markdown wrapping by LLM
 - Hallucinated fields
 - Case inconsistencies
 - Unsupported operators
-

Mitigation Techniques

- Output cleanup logic
 - JSON parsing validation
 - Controlled schema prompting
 - DSL validation layer
-

5.5 DSL Layer

Purpose

DSL (Domain Specific Language) acts as an intermediate representation between natural language and SQL.

Why DSL?

Direct NL-to-SQL is risky because:

- SQL injection risk
- Uncontrolled query generation
- Invalid queries
- Schema hallucinations
- Poor maintainability

DSL solves these by introducing a controlled abstraction layer.

DSL Structure

```
{
  "filters": [
    {
      "field": "pe_ratio",
      "operator": "<",
      "value": 20
    }
  ],
  "logic": "AND"
}
```

Extended DSL for Time Filtering

```
{
  "filters": [
    {
      "field": "revenue",
      "operator": ">",
      "value": 100000
    }
  ],
  "logic": "AND",
  "time_filter": {
    "type": "quarter",
    "value": "2024-Q2"
  }
}
```

5.6 DSL Validation Layer

Purpose

Ensures the generated DSL is safe and valid before SQL generation.

Validation Checks

Field Validation

Allowed fields:

- sector
 - pe_ratio
 - revenue
 - symbol
-

Operator Validation

Allowed operators:

- =
 - <
 -
 - <=
 - =
-

Logic Validation

Allowed logic:

- AND
 - OR
-

Time Filter Validation

Checks:

- quarter format
 - valid year
 - valid quarter values
-

Benefits

- Prevents invalid queries
 - Reduces SQL risk
 - Prevents hallucinated fields
 - Improves system stability
-

5.7 SQL Compiler Layer

Purpose

Convert validated DSL into parameterized SQL.

Core Responsibilities

- SQL generation
 - Field mapping
 - Join management
 - Parameter binding
 - Time filter conversion
-

Field Mapping Strategy

```
FIELD_MAPPING = {  
  "symbol": ("symbols", "s.symbol"),  
  "sector": ("symbols", "s.sector"),  
  "pe_ratio": ("fundamentals", "f.pe_ratio"),  
  "revenue": ("historical_metrics", "h.revenue")  
}
```

Query Compilation Example

DSL

```
{  
  "filters": [  
    {  
      "field": "pe_ratio",  
      "operator": "<",  
      "value": 20  
    }  
  ],  
  "logic": "AND"  
}
```

Generated SQL

```
SELECT s.symbol, s.company_name
FROM symbols s
JOIN fundamentals f ON s.id = f.company_id
WHERE f.pe_ratio < %s
```

Parameters

[20]

Safe SQL Strategy

The system uses parameterized queries.

This prevents:

- SQL injection
- Dynamic query vulnerabilities
- Unsafe concatenation

Quarter-Based Filtering

Implemented support for:

Q1, Q2, Q3, Q4

Converted internally into SQL-compatible quarter conditions.

5.8 Query Execution Layer

Purpose

Execute compiled SQL safely against PostgreSQL.

Responsibilities

- Database connection

- SQL execution
 - Parameter passing
 - Error handling
 - Result formatting
 - Empty result handling
-

Flow

```
Compiled SQL
↓
Database Driver
↓
PostgreSQL Execution
↓
Raw Rows
↓
JSON Formatting
↓
API Response
```

Error Handling

Handled scenarios:

- database failures
 - invalid queries
 - connection issues
 - empty result sets
 - execution failures
-

Response Formatting

Raw tuples are converted into dictionaries.

Example:

```
{
  "symbol": "AAPL",
  "company_name": "Apple Inc"
}
```

5.9 Database Layer

Database Used

PostgreSQL.

Database Tables

1. symbols

Stores:

- company id
 - company symbol
 - company name
 - sector
-

2. fundamentals

Stores:

- PE ratio
 - EPS
 - market cap
 - revenue growth
 - financial metrics
-

3. historical_metrics

Stores:

- quarterly data
 - revenue history
 - historical metrics
 - time-series financial data
-

4. users

Stores:

- user credentials
 - authentication data
-

5. portfolio

Stores:

- purchased stocks
 - quantities
 - user portfolio tracking
-

6. alerts

Stores:

- alert conditions
 - thresholds
 - alert metadata
-

6. Feature-Level Documentation

6.1 Natural Language Querying

Workflow

```
User Query
↓
Gemini Parser
↓
DSL JSON
↓
Validator
↓
SQL Compiler
↓
Execution Layer
↓
Results
```

Supported Queries

Examples:

- companies with pe_ratio < 20
- technology companies
- companies with revenue > 100000

- companies in Q2 2024
-

6.2 Portfolio Management

Features

- add stock
 - remove stock
 - view portfolio
 - investment calculations
 - profit/loss calculations
-

Calculations

Investment Value

$$\text{purchase_price} \times \text{quantity}$$

Current Value

$$\text{current_market_price} \times \text{quantity}$$

Profit / Loss

$$\text{current_value} - \text{investment_value}$$

6.3 Alerts System

Purpose

Notify users when specific financial conditions are met.

Alert Examples

- PE ratio > threshold
- EPS < threshold

- Revenue growth exceeds value
-

Alert Workflow

```
Create Alert
↓
Store Alert
↓
Evaluate Condition
↓
Trigger Alert
```

6.4 Quarter-Based Filtering

Purpose

Support time-aware financial analysis.

Example Query

```
Companies with revenue > 100000 in Q2 2024
```

Internal Conversion

```
2024-Q2 → SQL quarter condition
```

6.5 Growth-Based Filtering

Purpose

Analyze financial growth across time periods.

Logic

Uses SQL window functions to compare current and previous metrics.

Example

Revenue growth between quarters

7. API Documentation

7.1 Authentication APIs

Register

POST /auth/register

Login

POST /auth/login

7.2 Query APIs

Run Query

POST /query

7.3 Portfolio APIs

Add Portfolio Item

POST /portfolio

View Portfolio

```
GET /portfolio
```

7.4 Alerts APIs

Create Alert

```
POST /alerts
```

View Alerts

```
GET /alerts
```

8. Security Design

8.1 Authentication Security

- JWT tokens
 - Protected routes
 - Session validation
 - Password hashing
-

8.2 Database Security

- Parameterized SQL
 - Controlled DSL
 - Validation layer
 - Restricted operators
-

8.3 API Security

- Input validation
- Safe error messages
- Request validation

- Structured responses
-

9. Error Handling Strategy

Errors Handled

- invalid DSL
 - unsupported fields
 - unsupported operators
 - invalid authentication
 - database connection failures
 - empty results
 - malformed LLM outputs
-

Response Philosophy

The system avoids exposing raw database or internal server errors.

Safe JSON error responses are returned instead.

10. Caching Layer

Technology

Redis.

Purpose

Used for:

- reducing repeated DB calls
 - faster company list retrieval
 - performance optimization
-

Strategy

- cache query results
 - apply TTL expiration
 - reduce backend load
-

11. Testing Strategy

11.1 Unit Testing

Tested:

- DSL validation
 - SQL compiler
 - parser output
 - execution layer
-

11.2 API Testing

Tested using:

- Swagger UI
 - FastAPI interactive docs
-

11.3 Edge Case Testing

Scenarios tested:

- invalid DSL
 - empty results
 - malformed queries
 - unsupported filters
-

12. Performance Optimization

Optimizations Implemented

- Avoided SELECT *
 - Used parameterized SQL
 - Reduced unnecessary joins
 - Added Redis caching
 - Optimized query execution
-

13. Challenges Faced During Development

Major Challenges

1. LLM Inconsistency

LLM sometimes generated:

- invalid JSON
 - unsupported fields
 - malformed structures
-

2. Data Mapping Issues

Mismatch between:

- DSL fields
 - database schema
 - query compiler mappings
-

3. Integration Complexity

Connecting:

- frontend
- backend
- AI layer
- database
- authentication

was complex.

4. Time-Based Query Logic

Quarter-based filtering required:

- historical data handling
 - SQL date logic
 - time normalization
-

14. Future Enhancements

Planned Improvements

- Real-time stock APIs
 - ML-based recommendations
 - Advanced charting
 - Better NLP understanding
 - Multi-user scaling
 - Notification system
 - Cloud-native deployment
 - Full DevOps automation
-

15. DevOps Integration Readiness

The architecture is intentionally modular and API-driven to support future DevOps deployment.

Docker Compatibility

Each component can be containerized:

- frontend container
 - backend container
 - PostgreSQL container
 - Redis container
-

Kubernetes Readiness

Can be deployed as:

- backend deployment
 - frontend deployment
 - database services
 - ingress routing
 - autoscaling services
-

Jenkins Integration

Supports:

- CI/CD pipelines
- automated testing
- Docker image builds

- deployment automation
-

Terraform Readiness

Infrastructure provisioning can include:

- EC2/EKS
 - VPC
 - security groups
 - RDS PostgreSQL
 - monitoring stack
-

Monitoring Readiness

Prometheus and Grafana can monitor:

- API latency
 - query execution time
 - CPU usage
 - memory usage
 - request throughput
 - database performance
-

16. Project Folder Structure

```
backend/
├── routes/
│   ├── auth.py
│   ├── query.py
│   ├── portfolio.py
│   ├── alerts.py
│   └── companies.py
├── llm/
│   └── parser.py
├── dsl/
│   ├── schema.py
│   └── validator.py
├── execution/
│   └── query_executor.py
└── auth_utils.py
```

```
|— auth_dependency.py
|— cache.py
|— sql_compiler.py
```

```
frontend/
|— streamlit_app.py
```

```
main.py
```

```
database.py
```

17. End-to-End Example Flow

User Query

```
Technology companies with PE ratio less than 30
```

LLM Output

```
{
  "filters": [
    {
      "field": "sector",
      "operator": "=",
      "value": "Technology"
    },
    {
      "field": "pe_ratio",
      "operator": "<",
      "value": 30
    }
  ],
  "logic": "AND"
}
```

SQL Generated

```
SELECT s.symbol, s.company_name
FROM symbols s
```

```
JOIN fundamentals f ON s.id = f.company_id
WHERE s.sector = %s AND f.pe_ratio < %s
```

Parameters

```
["Technology", 30]
```

Final Output

```
{
  "results": [
    {
      "symbol": "AAPL",
      "company_name": "Apple Inc"
    }
  ]
}
```

18. Conclusion

The AI-Powered Stock Screener project demonstrates how AI systems can be integrated into backend architectures to simplify access to structured financial data.

The project combines:

- AI-based query understanding
- DSL abstraction
- safe SQL generation
- backend engineering
- database systems
- API design
- frontend integration

into a complete end-to-end intelligent financial querying platform.

The architecture is extensible and suitable for future DevOps integration, cloud deployment, containerization, CI/CD automation, and monitoring systems.